

Mockito Hands-On Exercises

CORRECTED TECHNICAL REFERENCE & IMPLEMENTATION BLUEPRINT

Overview of Key Testing Concepts:

- **Mocking:** Eliminates real-world external constraints (Network latency, Database writes, API limits) by substituting runtime objects with virtual structures.
- **Stubbing:** Controls execution behaviors by explicitly programming canned responses for targeted invocations.
- **Verification:** Assures correct system operation by auditing outbound messages, structural invariants, and interaction parameters.

Dependent Setup Infrastructure

To successfully compile and execute the exercises, the testing architecture requires a concrete target implementation. Below are the standard interface and service objects implicit within your mock specifications:

```
// 1. External Dependency Contract Interface
public interface ExternalApi {
    String getData();
}

// 2. Service Under Test (SUT) System Architecture
public class MyService {
    private final ExternalApi externalApi;

    public MyService(ExternalApi externalApi) {
        this.externalApi = externalApi;
    }

    public String fetchData() {
        return this.externalApi.getData();
    }
}
```

Exercise 1: Mocking and Stubbing

This exercise isolates the system behavior by stubbing out structural data. This ensures external environment stability does not corrupt local continuous integration validations.

Syntax Fix Applied:

The baseline document structural code contained an isolated `assertEquals` statement leaking beyond the boundary lines of the test method closure block. The solution below cleanly incorporates the assertion inside a single, safe execution context.

```
import static org.mockito.Mockito.*;
import static org.junit.jupiter.api.Assertions.assertEquals;
import org.junit.jupiter.api.Test;
import org.mockito.Mockito;

public class MyServiceTest {

    @Test
    public void testExternalApi() {
        // ARRANGE: Generate isolation proxy harness
        ExternalApi mockApi = Mockito.mock(ExternalApi.class);

        // STUB: Program specific execution route data behavior
        when(mockApi.getData()).thenReturn("Mock Data");

        // ARRANGE: Inject mock dependencies via constructor tracking
        MyService service = new MyService(mockApi);

        // ACT: Execute target runtime business logic transaction
        String result = service.fetchData();

        // ASSERT: Validate computed payload state matches expected stub criteria
        assertEquals("Mock Data", result);
    }
}
```

Exercise 2: Verifying Interactions

This exercise focuses on auditing interface call parameters and verification constraints. It checks behavior instead of structural state transitions to ensure dependent methods are triggered cleanly.

Syntax Fix Applied:

The original snippet had a closing bracket issue that left the `verify(...)` statement dangling outside the class structure. This has been compiled into standard structural alignment.

```

import static org.mockito.Mockito.*;
import org.junit.jupiter.api.Test;
import org.mockito.Mockito;

public class MyServiceInteractionTest {

    @Test
    public void testVerifyInteraction() {
        // ARRANGE: Create virtual dependency mock instances
        ExternalApi mockApi = Mockito.mock(ExternalApi.class);
        MyService service = new MyService(mockApi);

        // ACT: Run functional operations
        service.fetchData();

        // ASSERT / VERIFY: Audit that communication channels occurred properly
        // Validates that .getData() was called exactly 1 time on this specific mock
        verify(mockApi).getData();
    }
}

```